> **Info**
>
> This document uses Obsidian markdown, it may not render properly on some viewers

Newton-Raphsor Method

Theoretical development

The Newton-Raphsor Method relies on the Taylor polynomial approximation of of the function, this means that in order to apply this to find the roots of a function f, it needs to be a differentiable function of class 2, moreover, we want to algebraically manipulate it's Taylor expansion, so we'd like it's first derivative be different from 0 at the expansion point p_0.

$$f \in C^2, f'(p_0) \neq 0 \tag{condition}$$

The expansion is:

$$f(p) = f(p_0) + (p - p_0)f'(p_0) + \frac{(p - p_0)^2}{2} f''(\epsilon(p))$$

Since we want a root and $|p - p_0|$ is small (an assumption of the method) we get the approximation

$$0 \simeq f(p_0) + (p - p_0)f'(p)$$

From where some algebraic manipulations yield

$$p \simeq p_0 - \frac{f(p_0)}{f'(p_0)} \tag{recurrence relation}$$

This is a recursive algorithm for stipulating a root of f given a starting value.

> **Note**
>
> The idea of the algorithm now becomes:
> Apply the recurrence relation repeatedly until either p is within a tolerable error range, it reaches the maximum iteration limit, or $f'(p)$ becomes 0.

Building the algorithm

Now we actually build it. We'll use C++ 26 as it's the latest standard.
Let's start by defining our includes and constants:

```
#include <cmath>
#include <print>
#include <iostream>

// Tolerance
constexpr double EPS = 1e-6;
// Max iterations
constexpr size_t MAX_ITER = 256;
```

Here the constant "EPS" is our precision (10^{-6}) and "MAX_ITER" is the maximum amount of iteration we can run. Since the method converges rather quickly, this is ample time.
We now define some templates and types, this is standard generic programming stuff, as we want the method to accept any function.

```
// Real functions template
template<typename F>
concept RealFunction =
        requires(F f, double x)
        {
            { f(x) } -> std::convertible_to<double>;
        };

// Function type signature
using Func = std::function<double(double)>;
```

We want the following driver code:

```
// Functions to test
std::array<std::pair<Func, Func>, 3> functions = {{
    {
        [](const double x) { return x*x - std::cos(x); },
        [](const double x) { return 2*x + std::sin(x); }
    },
    {
        [](const double x) { return x*x*x - 1; },
        [](const double x) { return 3*x*x; }
```

```
    },
    {
        [](const double x) { return std::exp(x) - 10*x; },
        [](const double x) { return std::exp(x) - 10; }
    }
}};

std::println("Testing newton's method for functions:");
std::println("f(x) = x^2 - cos(x)");
std::println("f(x) = x^3 - 1");
std::println("f(x) = e^x - 10x\n");

// Testing
std::ranges::for_each(functions, [](auto&& fs) {
    double p0{};
    std::println("Give an initial guess p0");
    std::cin >> p0;

    try {
        double r{newton_raphsor(p0, fs.first, fs.second)};
        std::println("Newton's method returned {}\n", r);
    } catch (const std::domain_error &e) {
        std::print("Method failed because: {}\n\n", e.what());
    } catch (const std::runtime_error &e) {
        std::print("{}\n\n", e.what());
    }
}
```

This is good, scalable, code. We have a data structure representing our inputs, and a range-based for loop iterating it, properly handling errors on the method.

Now for the actual implementation:

```
// The method
// The method
template<RealFunction F, RealFunction DF>
auto newton_raphsor(const double p0, const F &f, const DF &f_prime) ->
double {
    size_t i{1};
    double p{}, pp{p0};
    while (i <= MAX_ITER) {
        const double fpp = f_prime(pp);

        // Probably diverged, we would get division by 0 (or near 0) here
        if (std::abs(fpp) < 1e-12)
```

```
            throw std::runtime_error("Derivative too small, possible
divergence");

        p = pp - f(pp) / fpp;
        std::println("Iteration {}: {}", i, p);

        // Converged
        if (std::abs(p - pp) < EPS or std::abs(f(p)) < EPS)
            return p;

        // Next iteration
        pp = p;
        ++i;
    }
    // No convergence
    throw std::runtime_error("Max iteration limit exceeded");
}
```

The function takes a starting point (initial guess) p_0 and any real function F along with it's derivative DF, returning either the calculated root or an error.
We follow the recurrence relation, only breaking if we get division by zero or exceed the iteration limit.

Analysis of the algorithm and conclusion

Here's some test data:

$$f_1(x) = x^2 - cos(x)$$
$$f_2(x) = x^3 - 1$$
$$f_3(x) = e^x - 10x$$

Test for f_1

Expected value: 0.82413231230252242296
Initial guess p_0: 1.2
Iteration 1: 0.8765817513780731
Iteration 2: 0.825579429156032
Iteration 3: 0.8241334876815659
Iteration 4: 0.8241323123032993
Newton's method returned: 0.8241323123032993

Test for f_2

Expected value: 1
Initial guess p_0: 1.2
Iteration 1: 1.031481481481481 4
Iteration 2: 1.0009510600585296
Iteration 3: 1.0000009033695991
Iteration 4: 1.0000000000000816
Newton's method returned: 1.0000000000000816

Test for f_3

Expected value: 0.11183255915896296483
Initial guess p_0: 1.2
Iteration 1: -0.09940643823654183
Iteration 2: 0.10944647923192224
Iteration 3: 0.11183220139852214
Iteration 4: 0.1118325591589549
Newton's method returned: 0.1118325591589549

Analysis

Notice that them all converged in just 4 iterations, which is great given that we gave it a limit of 256 iterations. Moreover, all approximations are within our range of 10^{-6} that we specified as expected.

Pitfalls

1. Requires the derivative of the function
2. Bad initial guess leads to diversion or finding a different, unexpected root

Conclusion

Obviously to do a real benchmark that would require many more functions, margins of error and initial guesses. However, that seams to be beyond the scope of this project.
Overall, the result were pretty satisfactory, as we achieved all proposed goals.